

Proyecto CS 2031 - DBP

La presente entrega corresponde a la Semana 7 del curso CS 2031 Desarrollo Basado en Plataforma, con fecha límite el viernes 25 de septiembre a las 11:59 p. m. Este hito representa la culminación del desarrollo del backend completo de nuestro proyecto, integrando todas las funcionalidades, medidas de seguridad, y pruebas necesarias para garantizar un sistema robusto y escalable. A continuación, se presenta la documentación detallada que abarca desde la concepción del problema hasta la implementación técnica, cumpliendo con los requisitos establecidos para esta entrega final del componente backend.

Entregables

Postman Collection (Semana 7)

Para complementar el informe, deberán proporcionar la documentación de su API utilizando Postman. Esta documentación debe ubicarse en la raíz del repositorio en un archivo **"postman_collection"** en formato JSON. Dentro de la colección, se deben incluir todos los endpoints necesarios para su proyecto, explicar su importancia y colocar ejemplos de funcionamiento. Además, las variables deben estar correctamente definidas y la autorización correctamente configurada (al igual que lo aprendido en la certificación de Postman).

Requisitos Informe (Semana 7)

El informe de backend se presentará en formato Markdown dentro del archivo **README.md**, ubicado en la raíz del repositorio de GitHub. Este deberá ser claro y bien estructurado, con una extensión de entre 1000 y 2,000 palabras. El informe incluirá las siguientes secciones.

Portada

- Título del Proyecto: Debe ser descriptivo y reflejar el propósito o la solución que proporciona el proyecto.
- Nombre del Curso: CS 2031 Desarrollo Basado en Plataforma.
- Nombres de los Integrantes: Incluir el nombre completo de cada miembro del equipo.

Índice

- Incluir una tabla de contenidos con las secciones del informe.

Introducción

- Contexto: Describir el contexto en el que surge la necesidad o problema a resolver.
- Objetivos del Proyecto: Detallar los objetivos específicos que el proyecto pretende alcanzar.

Identificación del Problema o Necesidad

- Descripción del Problema: Explicar en detalle el problema o necesidad del mercado que el proyecto busca abordar.
- Justificación: Por qué es relevante solucionar este problema o satisfacer esta necesidad.

Descripción de la Solución

- **Funcionalidades Implementadas:** Listar y describir las funcionalidades principales del proyecto, explicando cómo cada una contribuye a solucionar el problema o satisfacer la necesidad identificada.
- **Tecnologías Utilizadas:** Mencionar las tecnologías, lenguajes de programación, y herramientas empleadas en el desarrollo del proyecto, API externas, bases de datos.

Modelo de Entidades

- **Incluir un diagrama de entidades** (Entidad-Relación, Diagrama de clases, etc.) utilizado para diseñar la base de datos del proyecto.
- **Descripción de Entidades:** Explicar las entidades principales, sus atributos y las relaciones entre ellas.

Manejo de Errores

- **Manejo de Errores:** Explicar en términos generales las excepciones globales utilizadas y por qué se deben manejar.

Medidas de Seguridad Implementadas

- **Seguridad de Datos:** Explicar las técnicas y mecanismos adoptados para garantizar la seguridad de los datos (por ejemplo, cifrado, autenticación, gestión de permisos).
- **Prevención de Vulnerabilidades:** Describir las medidas tomadas para prevenir vulnerabilidades comunes (por ejemplo, inyección SQL, XSS, CSRF).

Eventos y Asincronía

- **Detallar los eventos utilizados**, explicar la importancia de su implementación en su proyecto, así como exponer el porqué deben ser asincrónicos.

GitHub & Management

- **Describir la manera en que se usó GitHub projects** o alguna otra herramienta para manejar los tasks (asignación de issues, deadlines, etc).
- **Describir el uso de GitHub Actions** y el flujo que implementaron para su proyecto en particular.

Conclusión

- **Logros del Proyecto:** Resumir los logros alcanzados con el proyecto en términos de resolver el problema o satisfacer la necesidad identificada.
- **Aprendizajes Clave:** Reflexionar sobre los aprendizajes más significativos obtenidos durante el desarrollo del proyecto.
- **Trabajo Futuro:** Sugerir posibles mejoras o extensiones para el proyecto.

Apéndices

- **Licencia:** Especificar la licencia bajo la cual se distribuye el proyecto.
- **Referencias**

Rúbrica

1. Entidades y Modelo de Datos (3 puntos)

1.1 Diseño de Entidades (1.5 puntos)

- **1.5 pto:** Más de 6 entidades correctamente definidas con atributos apropiados, tipos de datos adecuados, anotaciones JPA correctas (@Entity, @Table, @Column), y el modelo refleja completamente la lógica de negocio sin redundancias.
- **1.2 pto:** 5-6 entidades bien definidas con la mayoría de atributos y anotaciones correctas.
- **0.9 pto:** 4 entidades correctamente definidas con algunos atributos faltantes o anotaciones incompletas.
- **0.6 pto:** 3 entidades con definición básica pero funcional.
- **0.3 pto:** 2 entidades con múltiples problemas de definición.
- **0.0 pto:** Menos de 2 entidades o entidades mal definidas.

1.2 Relaciones entre Entidades (1.0 punto)

- **1.0 pto:** Todas las relaciones JPA que su aplicación requiera correctamente implementadas (@OneToMany, @ManyToOne, @ManyToMany, @OneToOne), con cascade types apropiados y fetch types optimizados (LAZY/EAGER según el caso).
- **0.8 pto:** La mayoría de relaciones correctamente implementadas con configuraciones apropiadas. Cascade types parcialmente correctos. Sin fetches optimizados.
- **0.6 pto:** Relaciones básicas implementadas, algunas configuraciones faltantes.
- **0.2 pto:** Relaciones implementadas con múltiples errores.
- **0.0 pto:** Sin relaciones o relaciones incorrectas.

1.3 Constraints y Validaciones (0.5 puntos)

- **0.5 pto:** Constraints apropiados a nivel de base de datos (@NotNull, @Unique, @Size, índices) y validaciones a nivel de aplicación (@Valid, @Min, @Max, @Email, @Pattern) correctamente aplicados.
- **0.3 pto:** Algunos constraints y validaciones implementados en entidades principales.
- **0.0 pto:** Sin constraints ni validaciones, o mal implementadas.

2. DTOs y Mapeo (2 puntos)

2.1 Definición de DTOs (1.2 puntos)

- **1.2 pto:** Más de 10 DTOs especializados (RequestDTO, ResponseDTO, CreateDTO, UpdateDTO, DetailDTO) con **separación clara de responsabilidades** y siguiendo el principio de segregación de interfaces.
- **1.0 pto:** Entre 8-10 DTOs bien organizados con **separación clara** entre request y response.
- **0.8 pto:** Entre 6-7 DTOs con buena organización o DTOs completos pero con pobre separación de responsabilidades.
- **0.6 pto:** Entre 4-5 DTOs con organización básica.
- **0.3 pto:** 3 DTOs o menos con organización inconsistente.
- **0.0 pto:** No utiliza DTOs, expone entidades directamente.

2.2 Mapeo Entidad-DTO (0.8 puntos)

- **0.8 pto:** Mapeo consistente y correcto en todos los endpoints usando mappers (ModelMapper, MapStruct, etc.) o métodos de conversión bien implementados. No hay fugas de datos sensibles.
- **0.6 pto:** Mapeo correcto en la mayoría de casos con implementación manual consistente.
- **0.2 pto:** Mapeo con múltiples problemas o inconsistencias.
- **0.0 pto:** Sin mapeo apropiado o mapeo incorrecto.

3. Arquitectura y Patrones de Diseño (2 puntos)

3.1 Separación de Capas y Responsabilidades (0.8 puntos)

- **0.8 pto:** Arquitectura en capas claramente definida (Controller → Service → Repository), cada capa cumple exclusivamente su responsabilidad, sin lógica de negocio en controllers, sin acceso directo a repositorios desde controllers, uso de interfaces para servicios cuando sea apropiado.
- **0.6 pto:** Separación de capas presente con cumplimiento de responsabilidades en la mayoría de casos, algunas violaciones menores (ej: validaciones complejas en controllers).
- **0.4 pto:** Separación de capas básica con violaciones notables (ej: algo de lógica de negocio en controllers, queries complejas en servicios).
- **0.0 pto:** Sin separación de capas o arquitectura desordenada.

3.2 Principio de Responsabilidad Única (SRP) (0.6 puntos)

- **0.6 pto:** Cada clase y método tiene una única responsabilidad bien definida, servicios enfocados (ej: UserService solo gestiona usuarios, no mezcla con lógica de otros dominios), métodos pequeños y cohesivos (**máximo 20-30 líneas**), nombres descriptivos que reflejan su propósito único.
- **0.5 pto:** La mayoría de clases y métodos cumplen SRP con algunas excepciones menores.
- **0.4 pto:** Cumplimiento parcial de SRP, algunos servicios o métodos con responsabilidades múltiples.
- **0.0 pto:** No aplica SRP, clases y métodos con múltiples responsabilidades sin separación.

3.3 Inyección de Dependencias y Bajo Acoplamiento (0.6 puntos)

- **0.6 pto:** Uso correcto de @Autowired e inyecciones por constructor, bajo acoplamiento entre módulos, sin uso de "new" para instanciar componentes de Spring.
- **0.5 pto:** Buena aplicación de inyección de dependencias con algunas oportunidades de mejora en acoplamiento.
- **0.4 pto:** Inyección de dependencias básica, algún acoplamiento alto entre componentes.
- **0.2 pto:** Uso inconsistente de inyección de dependencias o alto acoplamiento.
- **0.0 pto:** No utiliza inyección de dependencias apropiadamente o acoplamiento muy alto.

4. Manejo de Excepciones (2 puntos)

4.1 Excepciones Personalizadas (0.8 puntos)

- **0.8 pto:** Más de 7 excepciones personalizadas bien definidas y organizadas por categoría (ResourceNotFoundException, DuplicateResourceException, InvalidOperationException, UnauthorizedException, etc.), con jerarquía de excepciones si es apropiado.
- **0.6 pto:** Entre 5-7 excepciones personalizadas bien organizadas.
- **0.4 pto:** Entre 3-4 excepciones personalizadas básicas.
- **0.2 pto:** 2 excepciones personalizadas.
- **0.0 pto:** Solo excepciones genéricas o ninguna.

4.2 Global Exception Handler (1.2 puntos)

- **1.2 pto:** @ControllerAdvice implementado de manera completa manejando todas las excepciones personalizadas y de Spring (MethodArgumentNotValidException, HttpResponseMessageNotReadableException), con ErrorResponseDTO consistente incluyendo timestamp, status, error, message, path. Status codes HTTP correctos (400, 401, 403, 404, 409, 500).
- **1.0 pto:** @ControllerAdvice maneja la mayoría de excepciones con respuestas consistentes y status codes correctos.
- **0.5 pto:** Manejo global con algunas inconsistencias en formato o status codes.
- **0.0 pto:** Sin manejo global de excepciones.

5. Seguridad y Autenticación (4 puntos)

5.1 Configuración de Spring Security (1 punto)

- **1.0 pto:** Spring Security bien configurado con rutas y CORS funcionales. SecurityContext funcional y usado en los servicios protegidos.
- **0.6 pto:** Configuración funcional parcial con algunos problemas.
- **0.3 pto:** Configuración muy básica o con errores.
- **0.0 pto:** No implementa Spring Security.

5.2 Sistema JWT (1.5 puntos)

- **1.5 pto:** Implementación completa de JWT: generación de tokens en login, JwtAuthenticationFilter extrayendo y validando tokens del header Authorization, UserDetailsService personalizado, extracción de claims (userId, email, roles), validación de expiración, refresh tokens implementados, secret key en variables de entorno.
- **1.2 pto:** JWT funcional con generación, validación, y filter implementado correctamente.
- **0.6 pto:** Implementación parcial de JWT con problemas de validación.
- **0.3 pto:** Implementación muy básica o con errores significativos.
- **0.0 pto:** No implementa JWT.

5.3 Roles y Autorización (1 punto)

- **1 pto:** Sistema de roles completo con múltiples roles (USER, ADMIN, MANAGER, etc.), @PreAuthorize o @Secured en métodos sensibles, verificación de permisos en servicios, roles almacenados en BD y en token JWT.
- **0.6 pto:** Roles básicos implementados con verificación en algunos endpoints.
- **0.4 pto:** Sistema de roles muy básico.
- **0.2 pto:** Intento de implementación con problemas.
- **0.0 pto:** Sin manejo de roles.

5.4 Registro y Login (0.5 puntos)

- **0.5 pto:** Endpoints de registro y login completamente funcionales con validaciones (email único, password strength), password encoding con BCrypt, respuestas apropiadas con tokens.
- **0.3 pto:** Registro y login funcionales con validaciones básicas.
- **0.0 pto:** Registro/login con problemas o sin implementar.

6. API REST y Controllers (2 puntos)

6.1 Diseño RESTful (0.8 puntos)

- **0.8 pto:** Endpoints siguen convenciones REST completamente: URIs descriptivas (/api/v1/resources), recursos en plural, uso correcto de verbos HTTP (GET, POST, PUT, PATCH, DELETE), versionado de API (/v1/), HATEOAS considerado.
- **0.6 pto:** Mayoría de endpoints siguen convenciones REST con errores menores.
- **0.4 pto:** Diseño básico REST con algunas violaciones.
- **0.2 pto:** Diseño con múltiples problemas REST.
- **0.0 pto:** No sigue convenciones REST.

6.2 Códigos de Estado HTTP (0.7 puntos)

- **0.7 pto:** Uso correcto y consistente de todos los códigos HTTP apropiados: 200 (OK), 201 (Created), 204 (No Content), 400 (Bad Request), 401 (Unauthorized), 403 (Forbidden), 404 (Not Found), 409 (Conflict), 500 (Internal Server Error).
- **0.5 pto:** Uso correcto de la mayoría de códigos HTTP.
- **0.3 pto:** Uso parcial de códigos HTTP apropiados.
- **0.0 pto:** Uso incorrecto o inconsistente de códigos HTTP.

6.3 Estructura de Controladores (0.5 puntos)

- **0.5 pto:** Controladores delgados delegando lógica a servicios, uso correcto de anotaciones (@RestController, @RequestMapping, @PathVariable, @RequestParam, @RequestBody, @Valid), ResponseEntity utilizado apropiadamente, sin lógica de negocio en controllers.
- **0.3 pto:** Estructura básica correcta con algunas violaciones menores.
- **0.0 pto:** Controladores con lógica de negocio o mal estructurados.

7. Eventos y Asincronía (2 puntos)

7.1 Implementación de Eventos (1.0 punto)

- **1.0 pto:** Sistema de eventos implementado en más de 2 casos de uso usando @EventListener o @TransactionalEventListener, eventos personalizados definidos (ApplicationEvent), publishers y listeners correctamente configurados, desacoplamiento de componentes.
- **0.8 pto:** Eventos implementados en 2 casos de uso con buena configuración.
- **0.6 pto:** Eventos en 1 caso de uso además del correo.
- **0.3 pto:** Solo eventos básicos para correo.

- **0.0 pto:** No utiliza eventos.

7.2 Procesamiento Asíncrono (0.5 puntos)

- **0.5 pto:** @Async correctamente implementado con @EnableAsync, ThreadPoolTaskExecutor configurado, procesamiento asíncrono en múltiples servicios (correo, notificaciones, procesamiento de archivos, logs).
- **0.3 pto:** @Async implementado básicamente para servicios de correo/notificaciones.
- **0.0 pto:** No implementa asincronía.

7.3 Servicio de Correo Electrónico (0.5 puntos)

- **0.5 pto:** Servicio de email completamente funcional con JavaMailSender, Resend, etc. configurado, envío de emails HTML con plantillas (Thymeleaf u otros), confirmaciones de operaciones importantes (registro, recuperación de contraseña, confirmaciones de transacciones), manejo asíncrono, manejo de errores.
- **0.3 pto:** Servicio de email básico funcional para confirmaciones y sin el uso de plantillas.
- **0.0 pto:** No implementa servicio de email.

8. Deployment (2 puntos)

- **2.0 pto:** Backend desplegado en AWS usando ECS/EC2 + RDS con configuración completa: variables de entorno en producción, security groups configurados, base de datos en RDS, aplicación completamente funcional y accesible públicamente.
- **1.0 pto:** Backend desplegado en plataforma "Instant Deployment" (Railway, Render, Heroku) con base de datos en la nube y configuración apropiada.
- **0.5 pto:** Intento de deployment con la aplicación parcialmente funcional.
- **0.0 pto:** No desplegado.

9. GitHub y Documentación (1 punto)

9.1 README y Documentación (0.4 puntos)

- **0.4 pto:** README completo con: descripción del proyecto, tecnologías, instrucciones de instalación/ejecución local, variables de entorno requeridas, endpoints documentados, diagramas (ER, arquitectura), decisiones de diseño, equipo, link a deployment.
- **0.3 pto:** README con la mayoría de elementos, falta 1-2 secciones.
- **0.2 pto:** README básico con información mínima.
- **0.0 pto:** README ausente o muy deficiente.

9.2 Control de Versiones (0.4 puntos)

- **0.4 pto:** Excelente uso de Git: commits frecuentes y descriptivos, ramas por features/fixes (GitFlow), pull requests con code reviews, historial limpio sin archivos sensibles, .gitignore apropiado.
- **0.3 pto:** Buen uso de Git con commits descriptivos y uso de ramas.
- **0.2 pto:** Uso básico de Git con algunas buenas prácticas.
- **0.0 pto:** Mal uso de Git o prácticas deficientes.

9.3 Gestión de Proyecto (0.2 puntos)

- **0.2 pto:** Uso efectivo de GitHub Issues/Projects para organización de tareas, milestones, labels.
- **0.1 pto:** Uso básico de herramientas de gestión.
- **0.0 pto:** Sin uso de herramientas de gestión.

Notas Importantes

Convenciones y Buenas Prácticas:

- Nombres en inglés, descriptivos y consistentes.
- Código limpio sin comentarios innecesarios.
- Indentación y formato consistente.
- Variables de entorno para configuraciones sensibles.
- No commitar archivos .env, contraseñas, o API keys.

Elementos Bonus (pueden compensar puntos perdidos en otras áreas):

- Documentación Swagger/OpenAPI.
- Logging estructurado (SLF4J + Logback).
- Paginación en endpoints de listado.
- Filtros y búsquedas avanzadas.
- Upload de archivos a S3.
- Cobertura de tests >80%.
- Docker Compose para desarrollo local.
- CI/CD con GitHub Actions.

Nota sobre los puntos bonus:

- **No son transferibles a otras evaluaciones.**
- **La nota máxima final de la evaluación es 20. Cualquier puntaje excedente acumulado mediante los puntos bonus no será contabilizado.**